

FlakeHunter

An AI agent that finds flaky CI jobs, proves why, and opens a pull request to quarantine them.

EXASOL AI + DATA CHALLENGE 2026

Find

Identify flaky CI jobs using statistical patterns across 21,654 real CI runs

Prove

Generate and test hypotheses with SQL queries on Exasol — no invented evidence

Act

Open a pull request to quarantine the flaky job with database-backed evidence

The Problem: Flaky Tests Waste Hours

What is a flaky test?

A flaky test fails on **correct code** — not because of a real bug, but due to timing, infrastructure, or non-determinism. The code hasn't changed. The test result has.

The consequences compound

Wasted investigation time

Engineers spend ~1 hour chasing bugs that don't exist — re-running pipelines, bisecting commits, reading logs for a failure that was never real.

Alert fatigue sets in

Teams start ignoring red CI. Once red becomes the default, the signal is gone.

Real failures slip through

When nobody trusts the CI, genuine regressions go undetected. The cost of a flaky test isn't just the wasted hour — it's the real bug that hides behind it.

The Signal: Same Commit, Different Results

The core insight behind FlakeHunter is simple: **if the same commit both passes and fails, the code isn't the cause.** Flakiness is a property of the job, not the change.

Real example — pytorch/pytorch

One commit ran **84 times** in CI.

82

Passed

2

Failed

84

Total runs

Why this pattern matters

A job that fails 2 out of 84 times on the same commit is not broken — it is **flaky**. The distinction is critical:

Flaky pattern

pass → fail → pass — the same commit produces inconsistent results. The code is not the variable.

Broken pattern

pass → fail → fail — results are consistent after the change. A real regression has occurred.

Window functions in Exasol let FlakeHunter tell these two patterns apart across the full dataset.

How the Agent Works

FlakeHunter runs a structured investigative loop — generating hypotheses, writing SQL to test them, and refusing to state any number that isn't in a query result.

Triage

Identify candidate flaky jobs from **21,654 real CI jobs** across 3 open-source repos, using statistical patterns.

Probe

Write SQL for each hypothesis and execute it on Exasol via the MCP server's **24 tools** for SQL execution and schema introspection.

Loop

Refine the hypothesis and re-probe until the agent is confident or has exhausted the available evidence.

Hypothesize

Generate theories for the failure mode: timeout, race condition, infrastructure variance, or real regression.

Assess

Attempt to *disprove* the leading hypothesis — not confirm it. Failed queries can never count as evidence.

Conclude

Deliver a verdict with a **confidence score** grounded entirely in query results from the database.

A Real Investigation: `get_workflow_conclusion`

FlakeHunter investigated the `get_workflow_conclusion` job from pytorch/pytorch. The agent tested three hypotheses and ruled out two of them.

Runtime evidence

4.01s

Passing avg
158 successful runs

34.5s

Failing avg
2 failed runs

8.6×

Slower on fail
failing vs passing

An 8.6× runtime gap between passing and failing runs is a strong signal of a **timeout** — the job runs until it hits a wall, not until it completes.

Hypothesis scorecard

✓ Timeout

Supported. 8.6× runtime gap on failing runs.

× Runner infrastructure

Ruled out. Failures weren't tied to a particular runner type.

× Real regression

Ruled out. The same commit produced both pass and fail — code is not the variable.

Verdict

Flaky — confidence 0.85. Recommend quarantine via `continue-on-error: true`.

Why Exasol: Speed at Scale

FlakeHunter's investigative loop needs many SQL queries in rapid succession. Exasol's columnar engine makes this tractable — even for non-equi self-joins that generate hundreds of millions of comparisons.

0.07s

Flake triage

All jobs grouped by commit

0.34s

Self-join

13,331 jobs, ~178M pairs

21,654

CI jobs

home-assistant, pytorch, grafana

24

MCP tools

SQL execution + schema introspection

The self-join problem

To test whether failures cluster when CI is busy, every job is compared with every other job whose run window overlaps it. For 13,331 jobs that's about **178 million comparisons**. Exasol completes it in **0.34 seconds**.

Window functions for pattern detection

Exasol window functions let FlakeHunter tell the **pass→fail→pass** (flaky) pattern apart from **pass→fail→fail** (broken) across the full job history — without loading data into Python.

What We Got Wrong (and Fixed)

The first version of the agent was confidently wrong. It invented evidence, fabricated runner names, and reported failure rates from queries that had errored out.

The failure mode

Invented evidence

The agent fabricated runner names and failure rates that did not exist in the database.

Errored queries counted

Queries that returned errors were treated as supporting evidence. Confidence reached 0.95 from data that was never retrieved.

No schema grounding

The agent guessed column names and table structures, producing plausible-sounding but invalid SQL.

The three fixes

1 Failed queries are not evidence

A query that errors or returns no rows cannot be cited in the verdict. The agent must re-probe or acknowledge the gap.

2 Schema delivered upfront

The agent receives the real schema before it writes a single query. No guessing column names.

3 Numbers must come from results

The agent may not state any number that isn't in a query result. Every figure in the verdict traces to SQL output.

The Action: Quarantine via Pull Request

`flakehunter act` turns the agent's verdict into a concrete code change — a pull request on the user's own fork, never upstream.

What the PR does

Adds `continue-on-error: true`

The flaky job is quarantined: it still runs, but a failure no longer blocks the pipeline.

Evidence from the database

The PR's evidence tables are filled from query results, not AI-written text. Every number traces to SQL output.

Honest about scope

The PR states plainly that quarantine does not fix the root cause. It stops the job blocking the team while the owner investigates.

Safety constraints

Fork only

The agent opens PRs on the user's own fork. It never touches the upstream repository.

Minimal change

1 file changed, +3 lines. No refactoring, no collateral edits.

Human in the loop

The PR is a proposal. A human reviews and merges. FlakeHunter does not auto-merge.

Proof: Live Pull Request

FlakeHunter opened a real pull request on a fork of pytorch/pytorch. The change is minimal, the evidence is database-backed, and the PR is publicly visible.

Location

`github.com/mdfaheem23/pytorch/pull/1`

Change size

1 file, +3 lines — adds
`continue-on-error: true`
to the
`get_workflow_conclusion`
job.

Scope statement

The PR notes that quarantine masks the symptom; root-cause work is still required.

Evidence sourcing

The PR's tables — 84 attempts (82 passed, 2 failed) and 4.01 s vs 34.50 s average duration — are queried directly from Exasol, not generated by the AI. The PR came from a later run, with verdict confidence 0.95.

Screenshots: PR diff and evidence

Quarantine flaky job `get_workflow_conclusion` #1

Openmdfaheem23 wants to merge 1 commit into `main` from `flakehunter/quarantine-get_workflow_conclusion`

Conversation 0Commits 1Checks 0Files changed 1+3 -0

Changes from all commitsFile filterConversationsJump to

github/workflows/upload-test-stats.yml

@@ -40,6 +40,9 @@ jobs:

the conclusion field in the github context is sometimes null

solution adapted from https://github.com/community/community/discussions/21090#discussioncomment-3226271

get_workflow_conclusion:

+ # Quarantined by FlakeHunter: the same commit both passed and failed this job.

+ # Verdict confidence 0.95. Fix the cause, then remove this line.

+ continue-on-error: true

if: github.repository_owner == 'pytorch'

runs-on: ubuntu-24.04

outputs:

mdfaheem23 commented 17 minutes agoOwner

Quarantine `get_workflow_conclusion` — flaky in pytorch/pytorch

FlakeHunter found that the same commit both passed and failed this job, so the failures are not caused by the code under test.

Verdict: flaky · confidence 0.95 · Intermittent test failure on commit `64b9efd`

Evidence (queried from the Exasol warehouse)

commit	attempts	passed	failed
<code>64b9efd55</code>	84	82	2

outcome	runs	average duration
success	158	4.01 s
failure	2	34.50 s

What this changes

ReviewersNo reviews

AssigneesNo one assigned

LabelsNone yet

ProjectsNone yet

MilestoneNo milestone

DevelopmentSuccessfully merging this pull request may close these issues.

Lessons from Exasol Personal

Five concrete integration issues we hit during development — each resolved, each worth knowing for anyone building on the Exasol MCP stack.

1 **SQL execution off by default**
The MCP server ships with SQL execution disabled. `enable_read_query` must be switched on before the agent can run any query.

2 **Self-signed certificate**
Every MCP query fails with a generic "database error" until certificate validation is disabled.

3 **CSV HTTP import**
CSV import over HTTP can't reach the host from inside the VM. We load data with chunked INSERT instead.

4 **pyexasol placeholder syntax**
pyexasol uses `{name}` placeholders, not `:name`. Using `:name` fails with "Feature not supported: host parameter specification".

5 **CREATE OR REPLACE TABLE wipes data**
In a re-applied schema it drops and recreates the table, deleting existing data. Use `CREATE TABLE IF NOT EXISTS`.

Team Members

Mohammed Faheem K

Rakesh N

Sanjeev Kumar KS

Sanjay Ramaswamy

Gowrish

Vellore Institute of Technology